

BSC → PCS Connector — Sanal Gateway Cihaz Mimarisi (SPEC)

İş akışı aşaması: 1/6 — SPEC (AGENTS.md "Geliştirme İş Akışı — 6 aşama"). **Durum:** Tüm tasarım bu tek dokümanda — uygulama gününde TEK SEFERDE implement edilir (kullanıcı kararı 2026-09-15). **İlişkili:** [PCS-WATTOX-MIMARISI.md](#) (Wattox register seti — BMS bloğu B01-B23), [MANAGEMENT-SERVICE-MIMARISI.md](#).

1. Amaç ve Bağlam

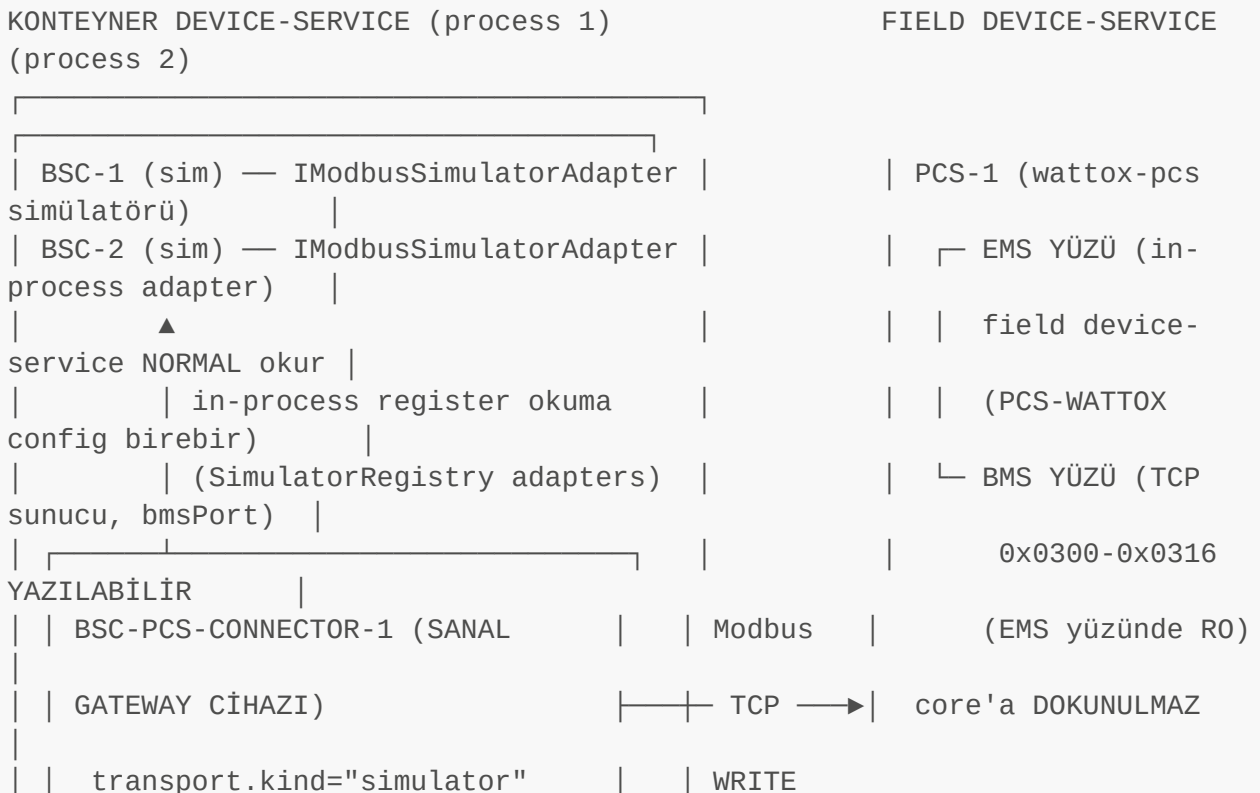
Gerçek senaryo: Konteyner içindeki BSC, PCS'e kendi linkinden (Modbus veya CANopen) otomatik veri doldurur — PCS'in BMS parametre bloğu (Wattox 0x0300-0x0316, B01-B23) BSC'den türemiştir. Bizim sistemimizde bu link YOK — iki katman ayrı device-service'lerde çalışır:

- **Konteyner device-service:** BSC simülatörleri (register-accurate) telemetri üretir.
- **Field device-service:** Wattox PCS simülatörü okunur/kontrol edilir (PCS-WATTOX config'i).

Sorun: PCS simülatörünün BMS bloğu BSC verisinden beslenmezse dummy kalır; field app'te PCS verisi tutarsız olur (ekran tünelden, veri değil — kullanıcı kuralı).

Çözüm: **BSC → PCS Connector** — simülatör katmanında çalışan, BSC'den okuduğu değerleri PCS simülatörünün BMS portuna Modbus üzerinden yazan **sanal gateway cihaz**. Gerçek donanımdaki BSC → PCS linkinin birebir simülasyonudur.

2. Mimari



```

type="bsc-pcs-connector"
├ in-process master: BSC reg.
├ dönüşüm (ratio/offset/bit)
├ outbound master: PCS BMS
└ kendi telemetrisi (link,
  sayaçlar, son yazım)

```

- Connector, konteyner cihaz envanterinde **gerçek bir cihaz** olarak görünür (UI'da link durumu, transfer sayaçları, son yazım zamanı izlenebilir).
- Tünel (ws-tunnel) PCS verisi TAŞIMAZ — connector kendi Modbus linkini kurar.
- Dev'de her iki stack aynı makinede → **target.host = 127.0.0.1**. Bu yapı YALNIZCA simülasyondur; üretim config'lerinde connector ASLA yer almaz (gerçekte BSC donanımı PCS'e kendisi yazar).

3. Purity Kuralları (ZORUNLU)

1. **device-service core değişmez** — connector yeni simülatör tipidir; tek dokunuş **SimulatorRegistry**'de 1 kayıt satırı (AGENTS kuralı: "yeni simülatör = modül + 1 kayıt").
2. **Field device-service değişmez** — BMS port sunucusu Wattox simülatörünün kendi yaşam döngüsü içinde açılır/kapanır (connect/disconnect).
3. **Sim-only her şey packages/simulators içinde** — connector modülü, BMS port sunucusu, mapping tipleri. Core/platform paketlerine connector kavramı GİRMEZ.
4. **Gerçek senaryoda kullanılmaz** — **transport.kind: "simulator"** olmadan connector var olamaz; üretim config'lerine girmez.
5. Tünel/kontrol mesaj protokolü bu özellik için GENİŞLEMEZ.

4. Bileşenler ve Sözleşmeler

4.1 BscPcsConnectorAdapter (packages/simulators/src/bsc-pcs-connector/)

IModbusSimulatorAdapter implemente eder — device-service onu diğer simülatörler gibi sarar (**SimulatorTransport**) ve config'deki telemetri girdilerini ondan okur.

Kendi register haritası (izleme):

Adres	İçerik
0x0000 (IR)	Link durumu: 0 kopuk / 1 bağlı (BMS port TCP)
0x0001 (IR)	Kaynak durumu: 0 tüm kaynaklar ok / 1 kısmi / 2 yok
0x0002-0x0003 (IR)	Son başarılı yazım zamanı (unix sn low/high)
0x0004-0x0005 (IR)	Toplam yazım sayacı (low/high)
0x0006-0x0007 (IR)	Başarısız yazım sayacı (low/high)
0x0008-0x000F	Rezerve

Yapılandırma (BscPcsConnectorConfig):

```
interface BscPcsConnectorConfig {
  mapping: BscPcsMapping; // mapping dosyası içeriği
  adapters: Map<string, IModbusSimulatorAdapter>; // registry'den BSC
  adapter'ları
  now?: () => number; // deterministik test
}
```

Tick davranışı (intervalMs'de bir):

1. Her **mapping** girdisi için kaynak adapter'dan değeri oku (in-process, **readInputRegister/readHoldingRegister**).
2. Dönüşüm: **raw_pcs = round(raw_bsc * ratio + offset)** (tamsayı çıktı — Modbus word).
3. Değişen veya **intervalMs**'den yaşlı değerleri toplu yazım olarak BMS port'a gönder (FC 0x10 write multiple registers; tek kayıt için FC 0x06).
4. Yazım sonucuna göre kendi izleme register'larını güncelle (link/sayaçlar/son yazım).
5. Kaynak okuma hatası → kaynak durumu 1/2; yazım hatası → link 0 + hata sayacı. Bağlantı kopuksa sonraki tick'te yeniden bağlanır (basit retry — her tick'te TCP bağlantı kontrolü).

Komut davranışı: Connector'a komut yazılmaz — kendi register'ları RO'dur (yazma girişimi yok sayılır).

4.2 BmsPortServer (packages/simulators/src/wattox-pcs/)

Wattox PCS simülatörünün içinde minimal Modbus TCP sunucu:

- **Port:** config'ten — **transport.bmsPort** (şema uzantısı, bkz. §6).
- **Kabul edilen fonksiyonlar:** FC 0x03 (read holding), FC 0x06 (write single), FC 0x10 (write multiple). Yazma YALNIZCA **0x0300-0x0316** aralığına; diğer adresler → exception 0x02 (invalid address).
- **Durum paylaşımı:** BMS bloğu tek depoda — EMS yüzü (in-process adapter) aynı depoyu RO okur; BMS yüzü aynı depoya yazar. İki yüz arasında tutarlılık bu depo ile garanti edilir.
- **Yaşam döngüsü:** **connect()** → sunucu açılır; **disconnect()** → kapatılır (SimulatorTransport lifecycle'ına bağlı — başka mekanizma YOK).
- **Bağlantı limiti:** 1 eşzamanlı istemci (connector); yeni bağlantı eskisini düşürür.

4.3 SimulatorRegistry erişimi (device-service — TEK değişiklik)

createFromConfigs sırasında üretilen adapter'lar **Map<deviceId, IModbusSimulatorAdapter>** olarak tutulur; **bsc-pcs-connector** fabrikasına (**deviceId**) => **adapter | undefined** sağlayıcısı verilir. Kayıt satırı:

```
this.registry.set("bsc-pcs-connector", {
  build: (deviceId, sim, elapsed) => {
    const mapping = parseBscPcsMapping(readFileSync(sim.registerMap!, "utf-8"));
    return {
      adapter: new BscPcsConnectorAdapter({ mapping, adapters:
this.adapters }),
```

```

    tick: () => connector.tick(elapsed),
  };
},
});

```

5. Mapping Config Formatı

Dosya: `bsc-pcs-mapping.json` (konteyner config-docker; connector cihazının `transport.registerMap` referansı).

```

{
  "target": { "host": "127.0.0.1", "port": 15502 },
  "intervalMs": 5000,
  "mappings": [
    // register→register: raw_bsc * ratio + offset → raw_pcs (tamsayı,
    // yuvarlama)
    { "kind": "register", "from": { "deviceId": "BSC-1", "table": "input",
    "address": 30055 }, "to": 772, "ratio": 10, "offset": 0 },
    // sabit: PCS register'ına sabit değer (B21 anma enerji vb.)
    { "kind": "constant", "to": 788, "value": 5018 },
    // bit→bit: kaynak register'ın N. biti → hedef register'ın M. bitine
    // (B02 status word türetme)
    { "kind": "bit", "from": { "deviceId": "BSC-1", "table": "input",
    "address": 30006, "bit": 0 }, "to": 769, "bit": 4 }
  ]
}

```

Kurallar:

- `from.table`: "input" (3x) veya "holding" (4x).
- `to` adresleri ondalık Wattox adresleri (0x0300=768 ... 0x0316=790) — BMS bloğu dışına yazım RED (BmsPortServer exception 0x02).
- `ratio/offset` ondalıklı sayı olabilir; çıktı `Math.round` ile tamsayıya indirgenir; S16 alanları için negatif değerler 16-bit twos-complement'e sığar.
- Mapping dosyası geçersizse connector **fail-fast** kurulmaz (açılışta `ValidationError` — device-service poll'u başlatmaz; konteyner açılış logunda görünür).
- Bilinmeyen `kind`/anahtar → red (strict parse).

6. Config Değişiklikleri

Dosya	Değişiklik
<code>shared-types</code> — <code>DeviceTransportConfig</code>	<code>bmsPort?</code> : <code>number</code> opsiyonel alan (Wattox PCS BMS sunucu portu; yalnızca <code>kind:"simulator"</code> + <code>type:"wattox-pcs"</code> için anlamlı)
<code>config-docker/bsc-pcs-connector-1.json</code> (konteyner)	Sanal gateway cihaz config'i: <code>type:"bsc-pcs-connector"</code> , <code>transport.registerMap</code> : "bsc-pcs-mapping.json",

Dosya	Değişiklik
	kendi izleme telemetrisi (§4.1 register haritası)
<code>config-docker/bsc-pcs-mapping.json</code> (konteyner)	B01-B23 eşleme (§7)
<code>config-field/pcs-1.json</code> (field)	<code>transport.bmsPort: 15502</code> eklenir

7. B01-B23 Tam Kapsam Eşleme Tablosu

Kaynak BSC register adresleri `bsc-1.json`'dan doğrulandı (scale sütunu BSC'nin kendi ölçeğidir; `ratio` ham → ham dönüşümdür):

Wattox	Ad	Adres	Kaynak (BSC, adres)	ratio	Not
B01	Pack çalışma durumu	768 (0x0300)	BSC State (30036)	1	durum kodları farklıysa mapping'de <code>offset</code> ile ötelenir — devreye alımda doğrulanır
B02	BMS status word	769 (0x0301)	bit türetme — bkz. §7.1	—	<code>kind:"bit"</code> kuralları
B03	Pack toplam voltaj (0.1V)	770 (0x0302)	BSC DC Voltage (30059)	1000	0.0001V → 0.1V
B04	Pack toplam akım (0.1A)	771 (0x0303)	BSC DC Current (30061)	100	0.001A → 0.1A
B05	Pack SOC (0.1%)	772 (0x0304)	BSC SOC (30055)	10	
B06	Pack SOH (0.1%)	773 (0x0305)	BSC SOH (30056)	10	
B07	Maks şarj akımı (0.1A)	774 (0x0306)	Max Current (30128)	100	
B08	Maksdeşarj akımı (0.1A)	775 (0x0307)	Max Current (30128)	100	
B09	Maks şarj gücü (0.1kW)	776 (0x0308)	Charge Power Limit (30063)	100	manevra vurgusu — dağıtımda kullanılır
B10	Maksdeşarj gücü (0.1kW)	777 (0x0309)	Discharge Power Limit (30065)	100	
B11	Maks hücre SOC (0.1%)	778 (0x030A)	Max SOC (30100)	10	
B12	Min hücre SOC (0.1%)	779 (0x030B)	Min SOC (30102)	10	

Wattox	Ad	Adres	Kaynak (BSC, adres)	ratio	Not
B13	En yüksek hücre sıcaklığı (0.1°C)	780 (0x030C)	Max Pack Temp (30149)	1	
B14	En düşük hücre sıcaklığı (0.1°C)	781 (0x030D)	Min Pack Temp (30151)	1	
B15	Maks hücre voltajı (0.001V)	782 (0x030E)	Max Cell Voltage (30138)	1	
B16	Min hücre voltajı (0.001V)	783 (0x030F)	Min Cell Voltage (30140)	1	
B17	Pack üst voltaj limiti (0.1V)	784 (0x0310)	Rack Max Voltage (30132)	1000	limit yaklaşımı — devreye alımda doğrulanır
B18	Pack alt voltaj limiti (0.1V)	785 (0x0311)	Rack Min Voltage (30134)	1000	
B19	Kullanılabilir şarj enerjisi (0.1kWh)	786 (0x0312)	EMU-1 Available Charge Energy	1	EMU register adresi devreye alımda netleşir
B20	Kullanılabilir deşarj enerjisi (0.1kWh)	787 (0x0313)	EMU-1 Available Discharge Energy	1	
B21	Anma enerji (0.001MWh)	788 (0x0314)	kind:"constant"	—	pil spesifikasyonundan sabit
B22	Pack SOP (0.1kW)	789 (0x0315)	Charge Power Limit (30063)	100	yaklaşım — devreye alımda doğrulanır
B23	DC tarafı sistem durumu	790 (0x0316)	BSC State (30036)	1	bit semantiği farklıysa bit türetme kurallarına taşınır

7.1 B02 — BMS Status Word bit türetme (taslak)

Hedef bit	Anlam	Kaynak (BSC bitfield adı)
bit0	BMS iletişim yok	BSC okuma hatası durumu (connector kaynak durumu)
bit1	BMS fault	BSC Alarm/Fault bitfield'ları (OR)
bit4	Şarj yasak	BSC ilgili alarm biti (devreye alımda eşlenir)
bit5	Deşarj yasak	BSC ilgili alarm biti
bit7	BMS alarm	BSC Alarm
bit8	Şarj+deşarj yasak	bit4 $\wedge$ bit5 (veya BSC genel block biti)

Bit kaynak adresleri devreye alım sırasında **bsc-1.json** bitfield listesinden netleştirilir — mapping dosyasında **kind:"bit"** girdileriyle ifade edilir; eşleşmeyen bitler 0 yazılır.

8. Yaşam Döngüsü ve Hata Kategorileri

Durum	Davranış
Connector connect()	BMS port'a TCP bağlantı kurar (ilk tick'te tembel bağlanma da kabul); tick döngüsü başlar
BMS port kapalı/PCS sim offline	link=0; yazımlar başarısız sayılır; her tick'te yeniden bağlanma denenir; konteyner UI'ında connector kartında görünür
Kaynak BSC adapter yok	Kaynak durumu=2; o mapping atlanır, diğerleri devam eder (kademeli bozulma)
Mapping dosyası geçersiz	Connector kurulmaz — ValidationError (fail-fast); device-service açılış logunda
BMS bloğu dışına yazım denemesi	BmsPortServer exception 0x02 — connector bunu hata sayacına işler
disconnect()	TCP kapatılır; tick durur (SimulatorTransport lifecycle)

9. Kabul Kriterleri

Kod	Kriter	Kanıt
K-C1	Connector config'i deviceConfigFileSchema 'dan geçer; bmsPort şema uzantısı testli	unit
K-C2	Mapping parse'ı strict; B01-B23 tablosu (§7) dosyada birebir	unit
K-C3	Dönüşüm: ratio/offset /yuvarlama + S16 negatif değerler doğru word üretir	unit
K-C4	kind:"bit" kaynak bit → hedef bit doğru yazar; eşleşmeyen bitler 0	unit
K-C5	BmsPortServer: FC 0x03/0x06/0x10 çalışır; BMS bloğu dışı adres → 0x02; EMS yüzü aynı depoyu RO okur (izolasyon)	unit
K-C6	Connector tick → TCP yazımı (fake socket ile) + sayaç/link register güncellemesi + kopuk bağlantıda retry	unit (fake timers)
K-C7	device-service core'da connector'a ait DAL YOK (purity — kod inceleme maddesi); SimulatorRegistry tek kayıt satırı	gözle + test
K-C8	Gözle: dev stack'te BSC sim değeri değişince PCS BMS bloğu (field device-service telemetrisi) güncellenir — dummy yok	gözle (E2E dev)
K-C9	Kapılar: yeni kod $\geq 70\%$ satır; dönüşüm/BmsPortServer $\geq 90\%$ branch	coverage

10. Görev Listesi

Görev	İçerik
T-C1	JSDoc + tipler: <code>BscPcsMapping</code> (strict parse), <code>BscPcsConnectorAdapter</code> , <code>BmsPortServer</code> sözleşmeleri (<code>packages/simulators</code>)
T-C2	<code>DeviceTransportConfig.bmsPort</code> şema uzantısı + test (<code>shared-types</code>)
T-C3	<code>bsc-pcs-mapping.json</code> + <code>bsc-pcs-connector-1.json</code> + <code>pcs-1.json</code> bmsPort (<code>config-docker/config-field</code>)
T-C4	<code>BmsPortServer</code> — minimal Modbus TCP sunucu (wattox-pcs içinde; BMS deposu EMS yüzüyle paylaşımlı)
T-C5	<code>BscPcsConnectorAdapter</code> — tick/yazım/izleme register'ları + retry
T-C6	SimulatorRegistry: adapter haritası + <code>bsc-pcs-connector</code> kaydı (tek satır)
T-C7	Testler: K-C1..K-C6 (kırmızı → yeşil)
T-C8	Dev compose/ortam notu (port 15502) + K-C8 gözle kontrolü
T-C9	DOGRULAMA + TEST-KAPSAMI + test-envanteri güncelleme

11. Aşama Eşlemesi (6 aşamalı iş akışı)

Aşama	Ürün
1. SPEC	Bu doküman
2. JSDoc	T-C1/T-C2 sözleşmeleri
3. TEST	T-C7 (kırmızı önce)
4. IMPL	T-C3..T-C6
5. SONUÇ	<code>BSC-PCS-CONNECTOR-DOGRULAMA.md</code>
6. KAPSAM	<code>BSC-PCS-CONNECTOR-TEST-KAPSAMI.md</code> + envanter

12. Açık Kararlar (devreye alımda netleşecek)

#	Konu
A1	B01/B23 durum kodu eşlemesi (BSC State ↔ Wattox pack running status) — offset veya bit türetme
A2	B02 bit kaynakları (BSC bitfield adları — <code>bsc-1.json</code> 'dan)
A3	B17/B18 limit kaynağı (Rack Max/Min Voltage uygun mu?)
A4	B19/B20 EMU register adresleri
A5	B22 SOP türetimi (min(charge,discharge) limiti mi?)

Konu

A6 Çift BSC durumunda kaynak seçimi (her konteynerde BSC-1/BSC-2 — MVP: mapping'de açık **deviceId**; agregasyon Faz 2)